

练习

5.4 是否需要再维护一个 full 指针，用于指向全满 slab 组成的链表呢？

5.1.6 常用的空闲链表

除了上述伙伴系统和 SLAB 分配器之外，还有其他基于不同空闲链表的内存分配方法。这些方法不仅可以用在内核态，也可以用在用户态的内存分配器中，比如堆分配器。本节主要介绍三种常见的空闲列表：隐式空闲列表、显式空闲列表和分离空闲列表。

第一种简单的空闲链表称为隐式空闲链表（Implicit Free List）。如图 5.7a 所示，链表里的每个元素代表一块内存区域，空闲（空白块）和非空闲的内存块混杂在同一个链表里。每个内存块头部存储了关于该块是否空闲的信息以及块大小信息。通过块大小，可以找到下一个块的位置。在分配空闲块的时候，分配器在这个链表中依次查询，找到第一块大小足够的空闲内存块即可返回。如果找到的空闲块大小不仅能够满足分配请求，还有足量剩余，则将该块进行分裂，一部分用于服务请求，剩余部分留用为新的空闲块，从而缓解内存碎片问题。在释放内存块的时候，为了尽可能避免外部碎片问题，分配器会检查该内存块紧邻的前后（根据块地址）两个内存块是否空闲，如果有空闲块存在，则进行合并，产生更大的空闲块。

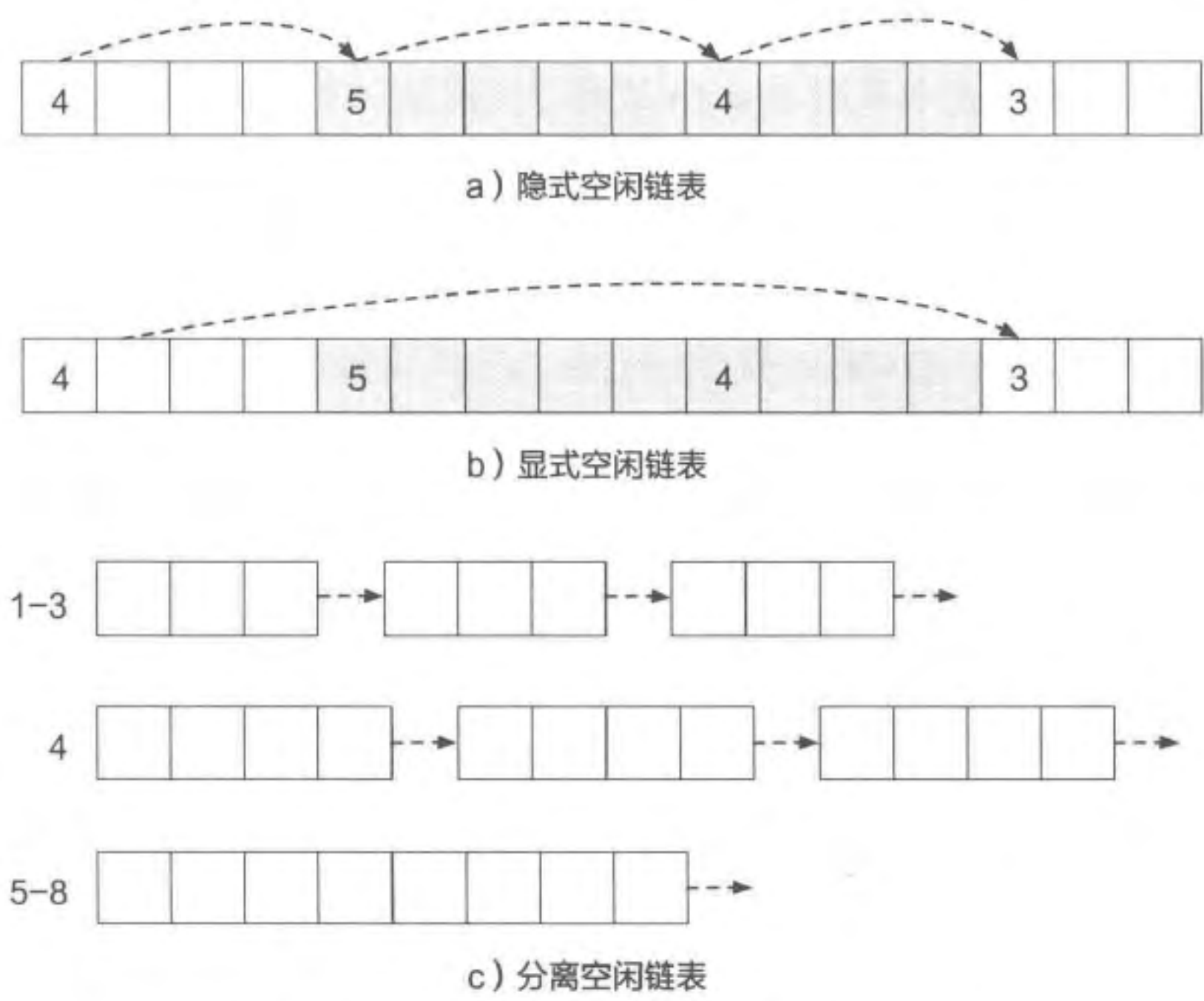


图 5.7 内存分配器中常用的三种空闲链表

与之很相似的另一种空闲链表称为显式空闲链表 (Explicit Free List)，如图 5.7b 所示。与隐式空闲链表最大的区别在于，显式空闲链表仅把空闲的内存块（而不是所有内存块）放在链表中。由于下一个空闲内存块可能存在内存中的任何位置，每个空闲块不能再依靠块大小找到下一个空闲块的位置，所以，除了块大小（合并块需要），每个空闲块需要额外维护两个指针（prev 和 next）来指向前后空闲块。不过，分配器仅需要在空闲块中维护指针，因此可以直接在空闲块的数据部分记录上述两个指针，无须使用额外的空间记录链表指针。显式空闲链表中内存块分配和释放的过程与隐式空闲链表中类似，不再赘述。相比之下，显式空闲链表在分配速度上具有优势，因为它的分配时间仅与空闲块数量成正相关，而隐式空闲链表的分配时间与所有块的数量成正相关。这个优势在内存使用率高的情况下更加明显，因为空闲块的数量更少而非空闲块更多。

另一种空闲链表数据结构称为分离空闲链表 (Segregated Free List)，是在显式空闲链表的基础上构建的。如图 5.7c 所示，其基本思想是维护多个不同的显式空闲链表，每个链表服务固定范围的分配请求，这一点和之前介绍的伙伴系统或 SLAB 分配器相似。分配内存块时，首先找到块大小对应的显式空闲链表，从中取出一个空闲块，若满足分配大小后有剩余，则将剩余部分插入相应大小的空闲链表中。如果在块大小对应的显式空闲链表中找不到合适的空闲块，则依次去更大的块大小对应的链表中寻找。释放块时，分配器依然可以先采用单个显式空闲链表中的合并策略，然后将合并产生的空闲块插入对应大小的空闲链表中。相比于普通显式空闲链表，分离空闲链表的优势在于能够获得更好的性能，一方面分配空闲块所需要的时间一般会更短，另一方面多个空闲链表可以更好地支持并发操作。此外，在分离空闲链表中采用 first-fit 策略（找到第一个空闲块即返回）能够近似地达到 best-fit 策略（最优策略，即找到大小最接近的空闲块）的内存利用率。如果分离空闲链表为每一种块大小都分配一个空闲链表，那么采用 first-fit 策略实际上也就是 best-fit 策略。

5.2 操作系统如何获得更多物理内存资源

本节主要知识点

- 物理内存不够用时怎么办？什么是换页机制？
- 如何选择页面进行替换？页替换的策略有哪些？
- 如何利用虚拟内存抽象节约物理内存（内存去重和内存压缩）？

5.2.1 换页机制

当操作系统需要为应用进程分配物理页时，若已经没有剩余的空闲物理页了，操作系统会束手无策吗？答案是否定的。假设如下场景：小明的笔记本配有 4GB 的物理内存，他一边使用 Photoshop 编辑大量高清图片——共需要占用 2GB 物理内存，一边玩英

英雄联盟——该游戏需要占用 3GB 物理内存。为什么两个总共需要 5GB 内存的应用进程能够同时运行在一台只有 4GB 物理内存的机器上呢？

操作系统设计了换页（Page Swapping）机制，基于虚拟内存与存储设备来实现上述需求。如图 5.8 所示，该机制的基本思想是当物理内存容量不够的时候，操作系统首先把若干物理页的内容写入类似磁盘这种容量更大且更加便宜的存储设备中，然后就可以回收这些物理页继续使用了。下面用一个简单的例子来说明换页具体要完成哪些步骤。当操作系统希望从应用进程 A 那里回收物理页 P（对应应用进程 A 中的虚拟页 V）时，操作系统需要将物理页 P 的内容写入磁盘上的一个位置，并且在应用进程 A 的页表中去除虚拟页 V 的映射，同时记录该物理页被换到磁盘上的对应位置。该过程被称为把物理页 P 换出（Swap Out），这样物理页 P 就可以被操作系统回收，并且分配给别的应用进程使用。

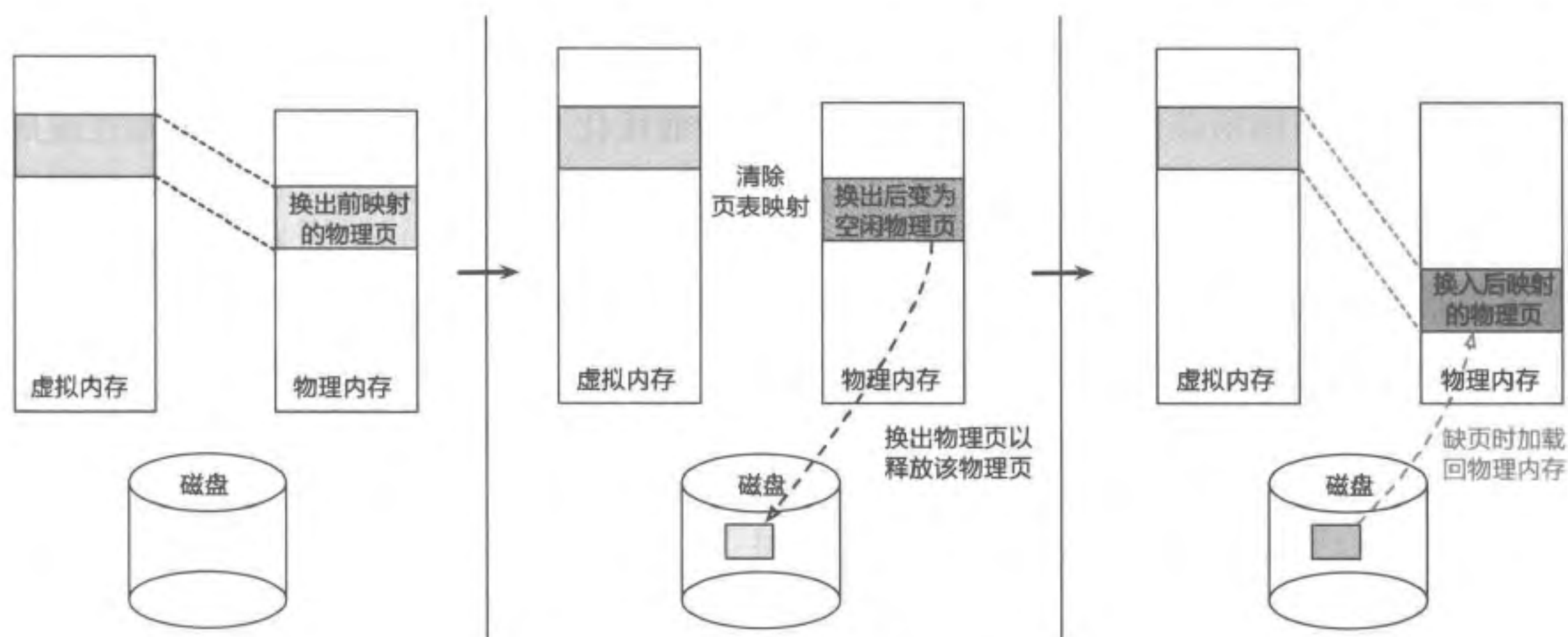


图 5.8 换页机制中的换出与换入

此后，当应用进程 A 访问虚拟页 V（对应物理页被换出）时，由于页表中没有映射，CPU 会触发缺页异常。操作系统的缺页异常处理函数会发现触发异常的虚拟地址（虚拟页）属于合法的虚拟地址范围，且对应的物理页被换出。因此它会分配一个空闲的物理页（若没有空闲的物理页，则需要通过换页的方式获取），将之前写入磁盘的数据内容重新加载到新分配的物理页中，并且在页表中填写虚拟页到该物理页的映射。这个过程被称为换入（Swap In）。之后，操作系统可以恢复应用进程的执行，即让应用进程从触发缺页异常的指令处继续执行。

利用换页机制，操作系统可以把物理内存中放不下的数据存放到磁盘上，等到需要的时候再放回物理内存中，从而能够在不修改应用进程的前提下提供超过物理内存实际容量的内存空间。因此，在所有应用进程的内存总需求大于物理内存总容量的场景下，操作系统通过换页机制能够获得比实际物理内存“看起来”更多的物理内存资源。

操作系统是用完所有物理页后才进行换页吗？这样的策略是可以的，但是在内存资

源紧张时，可能导致许多物理页分配操作都需要先进行换页，从而造成分配时延高的问题。因此，自然的想法是，操作系统可以设立阈值，在空闲的物理页数量低于阈值的时间内，操作系统便选择时机（比如系统较空闲时）进行换页操作，直到空闲页数量超过阈值。以 Linux 操作系统为例，操作系统设置三个阈值：高水位线（High Watermark）、低水位线（Low Watermark）、最小水位线（Min Watermark）。当空闲物理页数量小于低水位线时，则择机进行换页操作，目标是把空闲物理页数量恢复到高水位线；当空闲物理页数量低于最小水位线时，则立即进行换页操作，且是批量换出（具体数量可以按需配置）。

换页机制使得操作系统能够获取更多的物理内存资源，是否牺牲了什么作为代价呢？换页机制可能会带来性能损失。具体来说，当操作系统选择换出某个应用进程的页后，该应用进程再访问该页时会出现缺页异常，并且需要由操作系统从磁盘中把换出的页重新加载回内存中。由于磁盘操作是耗时的，所以应用进程的执行时长会增加，往往也意味着应用性能的下降。那么，有什么方法能够减少因换页机制带来的性能损失吗？操作系统可以使用预取机制（Prefetching）进行性能优化。预取机制的基本想法是在应用进程发生缺页错误前就完成换入过程，从而能够隐藏磁盘操作的时延，进而避免应用进程的性能下降。为实现预取机制，操作系统需要猜测应用进程在执行过程中需要哪些内存页，从而提前把这些页换入。一种可行的猜测方式是基于应用进程执行的空间本地性特征，即当应用进程访问某个内存页时，猜测相邻的下一个内存页也需要被访问。在这种猜测方式下，操作系统可以在需要换入应用进程的某个页时，将其地址空间中相邻的下一个内存页一并换入。若预取机制预测准确，则操作系统能够减少应用进程发生缺页异常的次数；反之，可能会拖累系统性能，因为做了不需要的换入操作，既占用磁盘读取带宽又占用内存资源。

小知识：预取机制的性能优势

预取命中能够避免性能损失的原因有两方面。第一，减少了缺页异常的发生，从而避免缺页异常处理过程本身消耗的时间。第二，减少磁盘操作花费的时间，对于许多磁盘来说，从磁盘中一次性加载两个内存页所花费的时间往往小于分别加载两个内存页所需的时间总和。

由于延迟映射同样会引起缺页异常，操作系统如何判断页表中没有映射是由于按需页面分配导致的还是换页导致的呢？操作系统需要标记进程中哪些虚拟页被换出以及换出的位置，若未被标记则说明尚未为该虚拟页分配物理页。一种高效的实现方法是：在页表项中做标记，以 AArch64 的最后一级页表项为例，MMU 仅根据页表项的第 0 位判断该映射是否存在（有效），操作系统可以利用其余位做标记以区别按需分配和换页两种情况。具体来说，若是按需分配，则说明此次缺页是由于应用进程第一次访问虚拟页导致的，该页表项从未被填写过，因此该页表项的内容为 0；而对于换页来说，操作系统

可以仅把页表项的第 0 位（最低位）设置成 0，并把页面换出位置记录在页表项的其余位中，从而使得发生缺页错误时，该页表项的内容非 0。

练习

5.5 应用进程地址空间中的虚拟页可能存在四种状态，分别是：

- (a) 未分配。
- (b) 已分配但尚未为其分配物理页。
- (c) 已分配且映射到物理页。
- (d) 已分配但对应物理页被换出。

请问当应用进程访问某虚拟页时，在上述四种状态下，操作系统分别会做什么？

5.2.2 页替换策略

当需要进行换页时，操作系统将根据页替换策略选择一个或一些物理页换出到磁盘以便让出空间。当已被换出的内存页再次被访问时，必须重新从磁盘换入物理内存，十分耗时。因此，页替换策略对性能具有较大的影响。

总的来说，页替换策略是依据硬件所提供的页面访问信息来猜测哪些页面应该被换出（比如短时间内再次被换入概率小的页面），从而最小化缺页异常的发生次数以提升性能。与此同时，操作系统也需要考量页替换策略本身的执行所带来的开销。不同的换页策略有其适合的应用场景，下面简单介绍一些经典的页替换策略。

MIN 策略 /OPT 策略

MIN 策略（Minimum 策略）又称为最优策略（Optimal 策略）。这一策略在选择被换出的页面时，优先选择未来不会再访问的页面，或者在最长时间内不会再访问的页面。该策略是理论最优的页替换策略，但在实际场景中很难实现。这是因为页访问顺序取决于应用进程，而操作系统通常无法预先得知应用进程未来访问页的顺序。该算法主要用来作为一个标准，衡量其他页替换算法的优劣。表 5.1 通过一个例子展示了这个策略的执行流程，假设物理内存中可以存放三个物理页，初始为空，某应用进程需要访问物理页面 1 ~ 5，访问顺序为 3、2、3、1、4、3、5、4、2、3、4、3。

表 5.1 MIN 策略执行流程

物理页访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
物理内存中存放的物理页	3	3	3	3	3	3	5	5	5	3	3	3
		2	2	2	2	2	2	2	2	2	2	2
				1	4	4	4	4	4	4	4	4
缺页异常（共 6 次）	是	是	否	是	是	否	是	否	否	是	否	否

FIFO 策略

FIFO（First-In First-Out，先入先出）策略是最简单的页替换策略，同时也正因为简单，所以其带来的时间开销很低。该策略优先选择最先换入的页进行换出。操作系统维护一个队列用于记录换入内存的物理页号，每换入一个物理页就把其页号加到队尾，因此最先换进的物理页号总是处于队头位置。当需要选择一个物理页面换出时，该策略总是选择位于队列头部的物理页号所对应的物理页。表 5.2 展示了同样的访存序列在 FIFO 策略下的表现。虽然该策略直观且开销低，但它在实际使用中表现往往不佳（因为页面换入顺序与使用是否频繁通常没有关联），因此也几乎不会被现代操作系统直接使用。

表 5.2 FIFO 策略执行流程

物理页访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
（该行是队列头部）	3	3	3	3	2	1	4	4	3	3	5	2
存储物理页号的 FIFO 队列		2	2	2	1	4	3	3	5	5	2	4
				1	4	3	5	5	2	2	4	3
缺页异常（共 9 次）	是	是	否	是	是	是	是	否	是	否	是	是

Second Chance 策略

Second Chance 策略是 FIFO 策略的一种改进版本。该策略的实现方法与 FIFO 策略类似，同样要求操作系统维护一个先入先出的队列用于记录换进物理内存的物理页号，此外还要为每一个物理页号维护一个访问标志位。如果访问的页面号已经处于队列中，则将其访问标志位置位。在寻找将要换出的内存页时，该策略优先查看位于队头的页号。此时可能有两种情况：如果它的访问标志位没有被置位，则换出该页号对应的内存页；若它的访问标志位已经被置位，则将该标志位清零，并将该内存页号挪到队尾（将其当成一个最近访问的内存页），并从新的队头开始重新寻找要换出的内存页。如果所有内存页号的访问标志位均已被置位，那么原本处于队头的内存页将在再次回到队头时被换出（彼时，其访问标志位已被清零）。通常情况下，由于考虑了页面访问的信息，Second Chance 策略会优于 FIFO 策略，若所有内存页的访问标志位均未被置位，那么 Second Chance 策略会暂时退化为 FIFO 策略。表 5.3 展示了同样的访存序列在 Second Chance 策略下的表现。

表 5.3 Second Chance 策略执行流程

物理页访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
（该行是队列头部）	3	3	3*	3*	1	1	3*	3*	3	3*	3*	3*
存储物理页号的 FIFO 队列		2	2	2	3	3*	4	4*	4	4	4*	4*
				1	4	4	5	5	2	2	2	2
缺页异常（共 6 次）	是	是	否	是	是	否	是	否	是	否	否	否

小知识：Belady 异常

操作系统分配给一个应用进程的物理页的数量会影响它在执行过程中发生缺页异常的次数。一般来说，对于同样的访存序列，物理页的数量越多，换页发生的次数越少。然而有的时候，更多的可用物理内存页面会导致更多的换页（和更低的性能），这种现象被称为 **Belady 异常**（Belady’s Anomaly）。该现象可能在操作系统使用 FIFO 和 Second Chance 等页替换策略时发生。本章结尾有一道思考题，要求构造一种物理页访问顺序，使得 FIFO 策略触发 Belady 异常。

LRU 策略

LRU（Least Recently Used）策略在选择被换出的页面时，优先选择最久未被访问的页面。该策略的出发点在于：过去数条指令频繁访问的页面，很可能在后续的数条指令中也被频繁访问。如果实际访问情况确实如此，LRU 策略能够提供接近于 MIN 策略的效果。该策略的一种实现方法是：操作系统维护一个链表，按照内存页的访问顺序将内存页号插入链表中（最久未访问的内存页号在链表头部，而最近访问的内存页在链表的尾部）；在每次内存访问后，操作系统把刚刚访问的内存页调整到链表尾部；每次都选择换出位于链表头部的页面。表 5.4 展示了同样的访存序列在 LRU 策略下的表现。在实际系统中，精确地实现这一策略需要时刻记录 CPU 访问了哪些物理页，其实现开销往往较大。

表 5.4 LRU 策略执行流程

物理页访问顺序	3	2	3	1	4	3	5	4	2	3	4	3
（该行是链表头部）	3	3	2	2	3	1	4	3	5	4	2	2
越不常访问的页号离头部越近		2	3	3	1	4	3	5	4	2	3	4
				1	4	3	5	4	2	3	4	3
缺页异常（共 7 次）	是	是	否	是	是	否	是	否	是	是	否	否

时钟算法策略

形象地说，时钟算法（Clock Algorithm）策略将换进物理内存的页号排成一个时钟的形状。该时钟有一个针臂，指向新换进内存的页号的后一个。同时，也为每一个页号维护一个访问标志位。每次需要选择换出页号时，该算法从针臂所指的页号开始检查。如果当前页面的访问标志位没有置位，即从上次检查到这次检查，该页面没有被访问过，将该页面替换；如果当前页面的访问标志位已被置位（即被访问过），那就将其访问标志位清空，并且针臂顺时针移动指针到下一个页面。如此重复，直到找到一个访问标志位未被置位的页面。时钟算法策略与 Second Chance 策略有相似之处，不过 Second Chance 策略需要将页号从队头移动到队尾，而时钟算法并不需要，所以后者的实现更加高效一些。

随机替换策略

顾名思义，在随机替换策略下，操作系统会任意挑选一个物理页面进行换出，该策

略的好处在于不需要维护页面的访问信息，不过相比于 LRU 等策略一般会引发更多的缺页。

5.2.3 页表项中的访问位与页替换策略实现

对于大多数页替换策略而言，操作系统都需要获得物理页是否被访问的信息，通常的实现是根据硬件自动设置的页表项中的访问位（如 4.1.4 节介绍的 Access Flag 和 Dirty State）来判断物理页是否被访问过。由于硬件是在地址翻译的过程中设置页表项中的访问位，所以操作系统需要维护物理页和页表项之间的关系，才能够通过页表项获取物理页是否被访问过的信息。具体来说，操作系统需要在页表中添加映射后（例如在代码片段 4.3 的 `add_mapping` 函数中）记录物理页对应的页表项，代码片段 5.5 给出了示意代码。

代码片段 5.5 记录每个物理页对应的页表项

```

1 // 在 physical_page 结构体中新增成员变量
2 struct physical_page {
3     ...
4
5     // 记录该物理页被映射到哪些页表项（称为反向映射）
6     list pgtbl_entries;
7 };
8
9 struct physical_page pages[NUM_PHYSICAL_PAGE];
10
11 void add_mapping(u64 pgtbl, u64 va, u64 pa)
12 {
13     ...
14
15     // 记录物理页被填写到哪个页表项
16     struct physical_page *page = &pages[pa/PAGE_SIZE];
17     add_one_entry(page->pgtbl_entries, pgtbl_entry);
18 }
19
20 // 参数是需要换出的物理页数量
21 void scan_and_swap(int num_page_to_swap)
22 {
23     int swap;
24
25     // 获取需要遍历的物理页区间（例如物理页号为 0 ~ 1000 的区间）
26     scan_range = get_scan_range();
27
28     while (num_page_to_swap != 0) {
29         for i in scan_range:
30             swap = 1;
31             list pgtbl_entries = pages[i]->pgtbl_entries;
32
33             // 利用 for_each_pte 宏遍历某物理页反向映射中的每个页表项
34             for_each_pte(pgtbl_entry, pgtbl_entries) {

```



```

35         if (is_accessed(pgtbl_entry)) {
36             // 将页表项中的访问位清零
37             clear_access(pgtbl_entry);
38             swap = 0;
39         }
40     }
41
42     if (swap) {
43         // 把第 i 个物理页换出
44         swap_out(i);
45         --num_page_to_swap;
46     }
47 }
48
49 tlb_flush();
50 }

```

由于共享内存等机制的存在，一个物理页可能被映射给多个虚拟页，即对应多个页表项，所以示意代码采用链表存放单个物理页对应的多个页表项。通过 `pages` 数据结构，操作系统可以遍历每个物理页对应的页表项，获取该物理页是否被访问过的信息。`scan_and_swap` 函数是换出若干物理页的简单实现，其根据物理页是否被访问过的信息（第 35 行），优先换出没有被访问过的物理页。第 37 行将访问位清零，否则无法区分仅访问过一次的页和真正频繁访问的页。当 TLB 中缓存地址翻译时，MMU 不会查找页表进行地址翻译，因而不会更新页表项中的访问位，那么在清空访问位后，该物理页即便后续被访问，相应的页表项中也不会被硬件设置为访问位，因此，第 49 行需要刷新 TLB 从而避免该情况发生。

小知识：反向映射机制

Linux 操作系统通过反向映射（Reverse Mapping）机制记录物理页对应页表项，实现更加完整且复杂，不过原理上与这里的简单实现类似。

5.2.4 工作集模型

在选择和实现页替换策略时，操作系统的原则是以最小的算法开销达到尽可能接近 MIN 策略的效果。然而，若选择的替换策略与实际的工作负载不匹配，则有可能造成颠簸现象（Thrashing），导致大部分 CPU 时间都被用来处理缺页异常以及等待缓慢的磁盘操作，而仅剩小部分的时间用于完成真正有意义的工作，因而引发严重性能损失。更糟糕的是，操作系统的调度器可能加剧颠簸现象。当 CPU 大部分时间都在等磁盘操作时，CPU 利用率大幅下降。为了提高 CPU 利用率，调度器会载入更多的应用进程，这又可能触发更多的缺页异常，进一步降低 CPU 利用率，进而导致连锁反应，引发严重的性能问题。

工作集模型 (Working Set Model) 能够有效地避免颠簸现象的发生。工作集的定义^[5]是：“一个应用进程在时刻 t 的工作集 W 为它在时间区间 $[t-x, t]$ 使用的内存页集合，也被视为它在未来 (下一个 x 时间内) 会访问的内存页集合。”该模型认为，应当将应用进程的工作集同时保持在物理内存中。因此，早期工作集模型的原则是 all-or-nothing，即一个应用进程的工作集要么全都在物理内存中 (运行时)，要么全都换出，从而减少该应用进程运行时发生换页的次数。现代操作系统很少直接采用 all-or-nothing 作为换页原则，但是工作集的概念依然指导着操作系统的换页策略，即优先将非工作集中的页换出。

那么如何高效地追踪工作集呢？一种常见的实现方法是工作集时钟算法。操作系统设置一个定时器，每经过固定的时间间隔，一个设置好的工作集追踪函数就会被调用。该追踪函数为每个内存页维护两个状态：上次使用时间和访问位，均初始化为 0。每次被调用时，该函数检查每个内存页的状态。如果访问位是 1，则说明在此次时间间隔内该页被访问，于是该函数会把当前系统时间赋值给该内存页的上次使用时间。该方法的前提是 CPU 硬件会在程序访问某个页的时候自动地将对应的访问位 (即页表项中的访问位，例如如图 4.6 所示的 AF 位) 设置成 1。如果访问位是 0，则说明在此次时间间隔内该页没有被访问，于是该函数会计算该页的年龄 (当前系统时间 - 该页的上次使用时间)。若该页年龄超过预设时间区间 x ，则它不再属于工作集。检查完一个页的状态之后，工作集追踪函数将其访问位设置为 0。通过工作集时钟算法或者具有相似设计的算法，操作系统能够有效地预测工作集，从而灵活地进行页替换。

5.2.5 利用虚拟内存抽象节约物理内存资源

内存去重

基于写时拷贝机制，操作系统进一步设计了内存去重功能。操作系统可以通过定期地在内存中扫描具有相同内容的物理页面 (全零页是一个典型的例子)，找到映射到这些物理页面的虚拟页面。然后只保留其中一个物理页，并将具有相同内容的其他虚拟页面都用写时拷贝的方式映射到这个物理页，然后释放其他的物理页面以供将来使用。该功能通常是操作系统主动发起使用，对于用户态应用进程完全透明。Linux 操作系统就实现了该功能，称为 KSM (Kernel Same-page Merging)。不过，内存去重功能会对应用进程访存时延造成影响，因为当应用进程写一个被去重的内存页时，既会触发缺页异常，又会导致内存拷贝，从而可能导致性能下降。

当内存去重发生在不同进程之间时，不同进程的虚拟页会映射到同一个物理页，也属于图 4.12 所示的共享内存情形。此外，内存去重也可以发生在同一个进程之中，此时该进程中的多个虚拟页被映射到同一个物理页。

小知识：内存去重的安全问题

内存去重虽然能够有效地节约物理内存资源，但是也可能带来安全问题^[6]。一

个简单的攻击是：攻击者可以在内存中通过穷举的方式不断构造数据，然后等待操作系统去重，再通过访问时延是否变长来窃取数据。比如假设攻击者知道某个进程在内存中保存了一个变量 secret，但不知道这个变量的值。于是攻击者首先构造一个除了变量 secret 外其他都一样的内存页，然后不断改变该变量的值，并记录访问该变量的时延，若时延突然变长，说明这个页发生了去重，即表示猜对了 secret 的值。一种防御这种攻击的可能方法是：操作系统仅在同一用户的应用进程内存之间进行内存去重，从而使得攻击者无法猜测别的用户应用进程中的数据。

内存压缩

为了节约内存资源，现代操作系统也引入了压缩算法来对内存数据进行压缩。其基本原理是：当内存资源不充足的时候，类似“换出”，操作系统选择一些“最近不太会使用”的内存页，压缩其中的数据（但不写入磁盘），从而释放出更多空闲内存。当应用进程访问被压缩的数据时，操作系统将其解压即可，所有操作都在内存中完成。相比于换出内存数据到磁盘上，这样的做法既能够更迅速地腾出空闲内存空间，又能够更快地恢复被压缩的数据。

如图 5.9 所示，Linux 操作系统支持的 zswap 机制是一个使用内存压缩技术的例子。zswap 在内存中为换页过程提供了缓冲区，称为 zswap 区域。操作系统将准备换出的内存数据进行压缩，并且将压缩后的内容写入 zswap 区域（实际上依然是内存）。通过

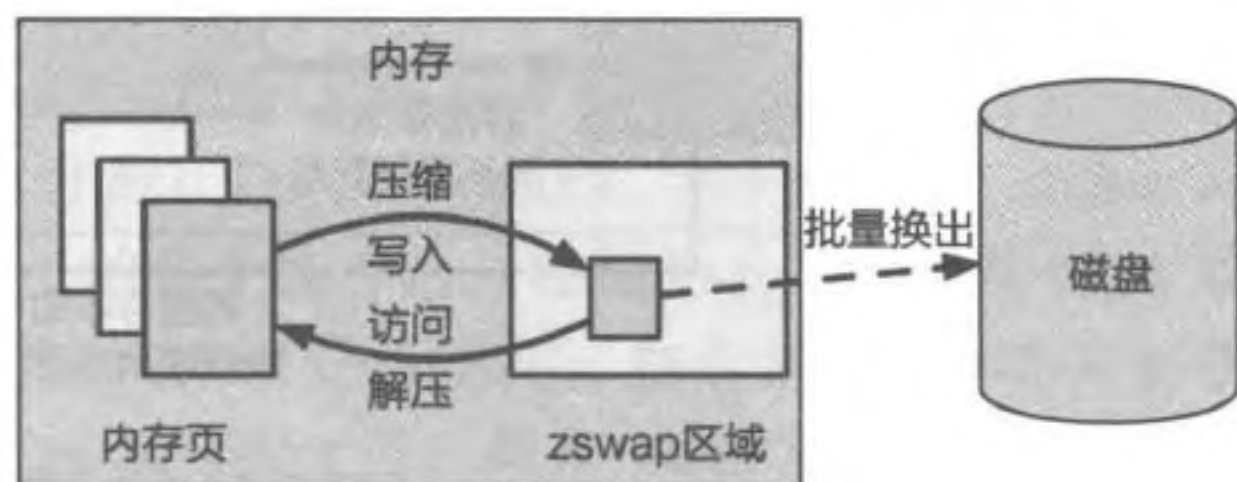


图 5.9 Linux 的 zswap 机制

过这样的设计，内存数据写到磁盘设备的操作可以延迟完成，从而有可能进行更加高效的磁盘批量 I/O，甚至可能避免磁盘 I/O。即使最后还是会触发磁盘操作，写出 / 读回的数据量由于经过压缩会明显减小。当被换出到 zswap 区域或者磁盘上的数据再次被访问时，该机制会把压缩的数据进行解压和换入。

5.3 性能导向的内存分配扩展机制

本节主要知识点

- 为什么需要 CPU 缓存？CPU 缓存的结构是怎样的？
- 在分配物理内存时操作系统需要考虑 CPU 缓存吗？
- 多核和 NUMA 对于内存分配有什么影响？